

매끄러움이 정확함을 이깁니다

이번 턴에 제 오류를 셋 잡으셨습니다. 셋의 뿌리는 하나입니다 — 저는 문서를 쓰는 자리에 있고, 문서에는 매끄러워지려는 힘이 있습니다. "ADMIN 0건"은 "ADMIN 1건(키 전달 대기)"보다 문장으로 낮고, "24종"은 "몇 종인지 모름"보다 읽기 좋습니다. 그리고 그 이긴 문장이 다음 판정의 근거가 됩니다.

D-322 · 판정자에게도 게이트를 건다

제 지시서의 모든 수치에 출처 태그를 강제합니다

제가 D-301에서 "건수 없는 게이트는 게이트로 인정하지 않는다"고 했습니다. 같은 규칙을 저에게 적용합니다 — 태그 없는 수치는 인용을 거부하고 「출처 미표기 N건」으로 돌려보내십시오. 제 문서는 저장소 밖이라 게이트를 걸 수 없으니 집행자를 너로 둡니다.

357

passed · 검증 20 · 혹 17

39절

구현 25 · 미착수 6 · 잡김 8

미측정 0

「부분」 칸 소멸

v2.9 → v3.0

132 → 138

제 오류 셋 · 그리고 잠금 술어

#	제가 쓴 것	무엇이 틀렸나
①	「어댑터 파일 존재 시 exit 1」	문언이 너무 넓다. 이미 정당하게 선 포트가 즉시 빨간불이 된다
②	「ADMIN 0건」	발급은 됐고 전달은 안 됐다. 나는 0이 되는 쪽으로 적었다
③	「게이트·검증기 24중」	"나는 그 수를 모른다"고 쓰고서 24라는 숫자를 적었다. 실측 27

셋의 뿌리는 하나입니다. 매끄러운 문장이 정확한 문장을 이겼습니다. 그리고 그 이긴 문장이 다음 판정의 근거가 됩니다 — D-292 이래 같은 병입니다.

D-322 지시서의 모든 수치·상태에 출처 태그

규약 신설 · 집행자는 Code

실측 네 보고서 화면에서 확인한 것 — 출처를 함께 적는다 · 추정 내가 어림한 것 — 네가 실측해서 알아 끼워야 하는 자리 · 인용 계약서·문서에서 그대로 옮긴 것

★ 강제 — 태그 없는 수치는 인용을 거부하라. 코드·대장·시험 어디에도 넣지 말고 회신에 「출처 미표기 N건」으로 돌려보내라. 이번 문서부터 적용했다 — 태그 없는 수가 보이면 그건 내 누락이다.

"발급된 것과 이 환경에 있는 것은 다른 사실이고, 그것이 이번 턴 D-316의 원칙입니다."

D-323 ADMIN 1건 유지 채택 — 잠금 술어를 blocked_now로

원칙 하강

★ 저는 D-316을 「계정에서 보았다」로 썼고, 너는 그것을 「이 환경에서 보았다」로 한 겹 더 내렸다. 원칙이 스스로 한 단계 내려간 것이다 — 채택한다. D-316 본문을 그 문장으로 교체하라.

술어: blocked_now = 「그것 때문에 지금 이 환경에서 막혀 있는가」. 발급 완료 ≠ 전달 완료 ≠ 환경 존재 — 세 사실을 한 칸에 두지 마라(D-290 계열). 「유형을 사실과 다르게 적으면 그 대장이 하는 일이 없어집니다」 — 네 문장이다. 대장 머리에 박아라.

강제 도구 — blockers.yaml 각 항목에 verified_at(이 환경에서 확인한 시각) · verified_by(무엇으로 확인했는가 — 파일·호출·화면). 두 칸이 비면 exit 1. "완료했다"는 진술이 아니라 확인 행위가 잠금을 내린다.

현재 잠김 실측 5건 — DEV 3 · CONTRACT 1(SDN 발송·대기) · ADMIN 1(juso 키 전달)

금지선은 파일이 아니라 행위에 낫는다

네 해석을 채택합니다. 포트·저하 운전·오류 계약은 명세 없이 확정 가능한 것이고(DA-02 S3), SDN에서 우리가 이미 그 경계를 그었습니다(D-300 부작위 시험). 문자 그대로면 정당하게 선 포트가 즉시 빨간불입니다.

금지선 낫는 법	술어	결과
✗ 파일 단위	adapters/juso/ 에 파일이 있으면	편하지만 같은 파일에 있는 정당한 것까지 죽인다
○ 행위 단위	외부 호스트로 나가는 호출이 있는가 응답 필드명을 참조하는 코드가 있는가	이 둘이 「스키마를 안다고 주장하는 행위」다. 포트 정의·타임아웃·풀백은 해당하지 않는다

이건 D-312(보안 지시를 수단이 아니라 금지 대상으로 쓴다)의 형제입니다 — 같은 병의 다른 얼굴. 저는 자꾸 "무엇이 있으면 안 되는가"를 파일·수단으로 적고, 정작 막아야 할 것은 행위입니다.

§ F-05 술어 정정도 같은 갈래

착시 ③ 표본으로 등재

✗ 처음 2단계 import 페포로 계산 → 662 중 98건 이 "진입면"으로 잡힘
delivery · stream_monitors 가 전이 import 만으로 걸렸다

✓ 정정 술어를 좁힘
"전이 근접성은 도달이 아니다"
옳다. 착시 ③(모수와 술어)의 표본으로 대장에 등재하라

§ D-326 · 자기표본 — 술어를 좁힐 때마다 반대편이 열린다

실측 27 **실측** · 7중 채워 10/27 · 나머지 17중은 D-311과 같은 이유로 래킷 — 옮습니다. 채택합니다.

★ 네가 든 사례를 정본으로 올립니다: verify_external_sources의 술어를 세 번 좁혔고, 세 번 다 반대편으로 넘어갔다가 자기시험이 그 자리에서 잡았습니다(산문 언급 → 문자열 통삭제 → 토큰 공백 결합).

원칙 — 술어를 좁힐 때마다 자기시험을 다시 돌린다. 좁히기는 한 방향으로만 가지 않습니다. 한 번 좁히면 반대편이 열립니다 — 이것이 D-310이 한 도구에서만 세 번 값을 한 이유입니다.

새로 만드는 도구는 자기표본 100% 의무. 래킷은 기존분에만 적용됩니다.

D-327 커밋 단위 술어 정밀화 — 게이트가 D-305를 교정했다

술어 갱신

D-305(2커밋)대로 나누려 했으나, pre-commit이 미스테인지분을 stash하는 순간 gx-tool-selftest가 "표본 없이 태어난 도구 2중"으로 빨개졌습니다.

★ 이걸 D-305를 부정한 것이 아니라 술어를 정밀하게 만든 것입니다. 갱신합니다 —

커밋 단위 = 함께 되돌려야 게이트가 초록인 최소 집합 (기준선과 그것을 재는 파일은 한 되돌림 단위 — 네 문장 그대로)

그리고 기록해 둡니다: 도구가 사람의 커밋 판단을 이겼습니다. D-286("절차는 도구로")의 도달점입니다. 앞으로 제 순서 지시와 게이트가 부딪히면 게이트를 따르고 사유를 적으십시오. 제 지시는 게이트보다 낮습니다.

절로 쪼개니 뿌리가 드러났습니다

"미착수 6절 중 4절이 표 두 개(임계값 · API Key 저장처)에 걸려 있습니다.
표 둘이면 잔여가 14 → 10입니다."

39

전체 절 실측

25

구현 (64%)

14 → 10

표 두 개로 줄어드는 잔여

0

미측정

14건짜리 대장에서는 절대 보이지 않았을 것입니다. 부분 5건은 서로 남남처럼 보였고, 쪼개고 나서야 같은 뿌리가 나왔습니다.
우리는 "얼마 남았나"를 알게 된 게 아니라 "어디를 누르면 여럿이 열리나"를 알게 됐습니다. 이것이 절 분해의 진짜 값입니다.

§1 표 ① 임계값(Threshold) — L3 커널

재난안전 전용이 아니다

착수 전 필수 — 지금 코드에 하드코딩된 임계값을 전수로 세십시오. 분모를 먼저 내십시오(D-301). 저는 그 수를 모릅니다 추정 없음 — 실측으로 답하십시오.

칸	내용
구조	항목 · 기본값 · 단위 · 적용 범위(전역/테넌트/카메라) · 변경 이력
지금 채우는 것	전역 기본만. 테넌트 override 자리는 두되 비운다 — Zone 폴리곤과 같은 대칭(D-299)
강제 도구	표 도입 후 코드에 남은 매직 넘버 임계값이 있으면 exit 1. 기존본은 이름으로 잠그고 새 것만 조인다 — 래칫(D-311). 소급 사유 요구 금지

§2 표 ② 자격증명 저장처 — K5 신뢰 커널

지금 키는 .env에 흩어져 있고, 외부 API가 늘면 관리가 무너집니다. 그리고 이번 juso 사건이 정확히 보여 준 것 — 「발급됐다」와 「환경에 있다」를 구분할 자리가 없었습니다.

서비스명 · 소유 테넌트 · 상태 · 마지막 확인 시각 · 접근 감사 로그
상태 : 'absent' | 'present' | 'verified' | 'rotated'
조회 : 언제나 마스크
★ 값은 저장소에 넣지 않는다 (D-204 · D-319).
표가 갖는 것은 값이 아니라 "있는가"의 사실이다.

이 표가 서면 D-323의 verified_at/verified_by가 손으로 적는 칸이 아니라 조회 결과가 됩니다.

★ 초효율 — 둘 다 L3 커널이므로 U1~U4 상용판이 같은 표를 씁니다. 이중 AC로 태깅하십시오. 계약 절 4개를 갇으면서 상용판 공통 기반이 함께 섭니다 — 「한 번 개발해서 두 번 판다」의 정확한 사례입니다.

실행 순서 — 표 둘이 먼저입니다

- 1 D-322~327 정서 — v2.9/132 → v3.0/138 · D-316 본문을 「이 환경에서 보았다」로 교체 · D-305 술어 갱신 D-323 · D-327
- 2 ★ 하드코딩 임계값 전수 — 표를 만들기 전에 분모부터, **실측**으로 건수를 답하라 D-301
- 3 ★ 표 ① 임계값 커널 (L3) — 전역 기본만 · 테넌트 override 자리는 비움 · 매직 넘버 래치 게이트 D-325
- 4 ★ 표 ② 자격증명 저장처 (K5) — 상태 4값 · 마스크 조회 · 감사 로그 · 값은 저장하지 않는다 D-325 · D-204
- 5 ③④ 완료 시 미착수 6절 → 2절 · 잔여 14절 → 10절 확인하고 절 표를 다시 내려 이번 턴의 결과물
- 6 blockers.yaml에 verified_at/verified_by 추가 — ④가 서면 조회 결과로 채워진다 D-323
- 7 새 도구 자기표본 100% · 기존 17종 래치 유지 D-326
- 8 juso 키가 환경에 도착하면 probe_juso_direction.py 1회 — 논블로킹. 없으면 ㉔ 유지하고 넘어가라 D-318
- 9 여기서 멈춘다. 잔여 10절 표를 보고 최종 관문(릴리스 후보 → 통합시험)을 정의한다 보고 후 편집

※ VIDEO 구간 추출 · ZONE 폴리곤은 잔여 10절 안에 있습니다. ⑨에서 순서를 정합니다.
 ※ 기상특보·특일·학교위치 3종 스키마 실측은 ④ 이후 — 표 ②가 서면 키 상태 조회가 그 실측의 전제가 됩니다.

§ 대표개 — 5분짜리 1건, 그리고 그것이 마지막 ADMIN입니다

Code가 제 표를 정정했습니다. 제가 「ADMIN 0건」이라 적었는데, **승인키가 발급된 것과 개발 환경에 있는 것은 다른 사실**입니다. 지금 개발 환경에 juso 키가 없어 실호출 판정이 서지 못했습니다.

- 1 C:\GuardianX-vault\keys\ 폴더를 만드십시오 (없으면)
- 2 그 안에 juso.env 파일을 만들고 두 줄만 적으십시오 — 키 값만 붙여넣기, 다른 설명 불필요
- 3 저장하고 Code에게 "juso 키 넣었다"고만 알려 주십시오. 나머지는 Code가 합니다

```
# C:\GuardianX-vault\keys\juso.env - 저장소 밖. 커밋되지 않습니다
JUSO_API_KEY_1=<신청서 A 승인키>
JUSO_API_KEY_2=<신청서 B 승인키>
```

이 1건이 끝나면 우리 손 밖에 남는 것은 SDN 하나입니다. Code는 값을 읽어 쓰기만 하고 어디에도 적지 않습니다.
 그리고 재발급 — 키가 하면 캡처로 대화에 남았습니다. 아직 아무 코드도 그 키에 묶여 있지 않으니 지금이 가장 짤 시점입니다. 재발급하시면 새 키를 위 파일에 적으시면 됩니다.